Laboratorio 2:
Análisis de microarquitecturas en procesadores RISC-V

Objetivos

- Utilizar una herramienta de simulación que permita analizar la performance de un microprocesador con distintas microarquitecturas.
- Analizar cómo impactan en la performance de un microprocesador variaciones en los tamaños y características de la caché y en el predictor de saltos.
- Comparar el rendimiento de un microprocesador con ejecución en orden respecto a uno fuera de orden.
- Observar, instrucción por instrucción, cómo avanza un programa por el pipeline.

Condiciones

- Realizar el trabajo práctico en grupos de **4 personas**, respetando los grupos del lab 1.
- La fecha límite de entrega es el **martes 17 de noviembre a las 23:59**, a través del repositorio de GitHub asignado. No se aceptan trabajos presentados después de esta fecha.
- Se verificará que todos los integrantes hayan aportado al trabajo a partir de los commits realizados en el repositorio asignado.

Formato de entrega

- Debe entregarse mediante el repositorio de GitHub, creando un tag que indique la versión final de cada ejercicio: "tag_ej1" y "tag_ej2" respectivamente.
- En el repositorio deben estar los códigos en assembler, sin modificar los nombres de los archivos, y las configuraciones de los procesadores que se utilicen. En el archivo `Informe.md` se debe escribir el informe con las partes solicitadas en cada ejercicio.
- No subir al repositorio los archivos que genera la simulación, el gitignore dado debería ignorar todos estos archivos (`se_results`, `sim.log`, `minorflow`, `konata` y los `.img`). Las capturas de los visualizadores que se incluyan en el informe sí deben subirse.
- No está permitido compartir código o informes entre grupos.
- No está permitido subir el código a repositorios públicos.

Calificación

El ejercicio 1 es obligatorio: su resolución debe estar aprobada para obtener la regularidad de la materia. El ejercicio 2 es opcional, y es condición necesaria para acceder a la promoción.

Desarrollo

La realización del presente laboratorio pretende que el alumno sea capaz de comparar y analizar el impacto de modificar características de la microarquitectura de un procesador en la velocidad

de ejecución de distintos códigos. Para esto se utiliza el simulador **gem5** con dos procesadores RISC-V (RV64GC):

- **In-order**: un modelo de **CVA6**, un procesador RISC-V real, de código abierto, con un pipeline en orden de 6 etapas. El modelo de gem5 fue calibrado contra el diseño real del procesador, de modo que la cantidad de ciclos que reporta la simulación es representativa de la del CVA6.
- **Out-of-order**: el procesador fuera de orden genérico de gem5 (O3CPU), con tamaños de colas y anchos de emisión similares a los de un ARM Cortex-A15.

El simulador se provee instalado en un contenedor Docker. Los archivos del repositorio se organizan en:

- `benchmarks`: los templates de los ejercicios, en assembler RISC-V.
- `scripts`: la configuración de la simulación. `cpu_config.toml` tiene las características generales del sistema (frecuencia y memoria principal), e `in_order.py` y `out_of_order.py`, las de cada procesador: el tamaño y las vías de las cachés y el predictor de saltos.
- `visualizadores`: los visualizadores de la ejecución en el pipeline, **MinorFlow** para el procesador in-order y **Konata** para el out-of-order.

Cada simulación se corre con `run_simulation.sh`, que muestra las métricas de la ejecución, genera la traza y abre el visualizador que corresponde. Cómo instalar Docker, correr las simulaciones y crear nuevas configuraciones de procesador se detalla en el archivo `README.md`.

Las métricas que muestra la simulación son las mismas que muestra MinorFlow, con las mismas definiciones: un *miss* es un acceso que tuvo que traer una línea de memoria, y los *hits* son los accesos que no fueron misses. Corresponden solo a la región del código entre `CODE HERE` y `END CODE` de los templates.

Ejercicio 1

DAXPY es la rutina principal del benchmark LINPACK, utilizado para medir la performance de microprocesadores, principalmente por su alta carga de cálculo de punto flotante. Sea *alpha* una constante, y X, Y y Z vectores de longitud N (todos ellos valores en punto flotante), el cálculo se realiza como se muestra a continuación:

```
const int N;
double X[N], Y[N], Z[N], alpha;

for (int i = 0; i < N; ++i)
{
    Z[i] = alpha * X[i] + Y[i];
}
```

Considerando que $N = 4096$, se pide:

a) Escribir el código del microbenchmark DAXPY en assembler RISC-V en el template `ej_daxpy.s`. Verificar su funcionamiento con MinorFlow: que el loop se ejecute N veces y que cada iteración haga los accesos a memoria esperados.

b) Correr la simulación en el procesador in-order para los siguientes tamaños de caché de datos: 8 KiB, 16 KiB y 32 KiB, con mapeo directo (1 vía), y obtener las siguientes métricas: cantidad de ciclos (*Cycles*), misses y hits de la caché de datos (*D-cache misses* y *D-cache hits*, separados en load y store) y su miss rate.

c) Repetir el punto b) para cachés asociativas por conjuntos de 2, 4 y 8 vías (parámetro `assoc` de la caché de datos).

d) Graficar las métricas obtenidas en los puntos b) y c), y analizar y justificar los resultados respecto al código escrito en el punto a). Es muy importante analizar el código del punto a) y explicar en base a él los resultados obtenidos. En particular:

- Calcular a qué conjunto de la caché corresponden $X[i]$, $Y[i]$ y $Z[i]$ en cada configuración, y cuántas líneas como mínimo tiene que traer de memoria el programa.
- Mostrar con MinorFlow, para una misma instrucción del loop, la diferencia entre la ejecución con 1 vía y con 8 vías, y explicarla.

e) Reescribir el código utilizando técnicas estáticas de mejora (loop unrolling, reordenamiento de instrucciones, etc.) para que, con una caché de datos de 32 KiB y mapeo directo, se obtenga un rendimiento similar al de 2 vías. Correr también el código optimizado con 8 vías y comparar con el código original: ¿mejora en todos los casos? Explicar por qué.

f) Correr ambos códigos en el procesador out-of-order con la caché de datos utilizada en el punto e), y comparar con los resultados de los puntos c) y e). ¿Cuánto mejora el código optimizado en cada procesador? Usar Konata para explicar las diferencias.

En el informe se deben incluir los gráficos del punto d) con su análisis, una breve explicación de las técnicas de mejora aplicadas en el punto e) y la comparación del punto f), con las capturas de los visualizadores que justifiquen el análisis. Se deben entregar los códigos assembler de los puntos a) y e).

Ejercicio 2

En este ejercicio se analiza la diferencia entre dos predictores de saltos: el **predictor de 2 bits (local)**, que es el que tiene CVA6, y el **predictor por torneos** (similar al del procesador Alpha 21264), compuesto por un predictor local y uno global. En los archivos de procesador corresponden a `BP_local_2bits` y `BP_tournament`.

Se utiliza como benchmark la función de activación **ReLU6**, la más usada en redes neuronales cuantizadas: $\text{ReLU6}(x) = \min(\max(0, x), 6)$. El código calcula, para cada una de las N salidas de una capa, la suma de las activaciones ReLU de sus 4 canales:

```
const int N;
signed char x[N][4];

for (int i = 0; i < N; ++i)
{
    for (int c = 0; c < 4; ++c)
    {
        if (x[i][c] < 0)
            x[i][c] = 0;
        if (x[i][c] > 6)
            x[i][c] = 6;
    }
}
```

El template `ej_branchPred.s` trae dos conjuntos de activaciones, con $N = 256$: `activ_ruido`, valores al azar como los de una red sin entrenar, y `activ_imagen`, las activaciones de una imagen con zonas uniformes. El conjunto a utilizar se elige en la línea `.set ACTIVACIONES` del template. Se pide:

a) Escribir el código en assembler RISC-V en el template `ej_branchPred.s`, respetando el código C: la ReLU se debe implementar con un salto condicional, sin usar la instrucción `max`.

Antes de simular, identificar los saltos condicionales del código y deducir, para cada uno y con cada conjunto de activaciones, la secuencia de resultados (tomado / no tomado) y el miss rate esperado con cada predictor.

b) Correr el código en el procesador in-order con cada predictor y cada conjunto de activaciones, y obtener la cantidad de ciclos y el miss rate, calculado como:

$$\frac{\text{Mispredicted} + \text{Unpredicted}}{\text{Branches}}$$

Comparar con lo deducido en el punto a), y usar MinorFlow para identificar qué saltos fallan con cada predictor.

c) Escribir una segunda versión en el archivo `ej_branchPred_sinsalto.s` que calcule la ReLU sin salto condicional (por ejemplo, con la instrucción `max`). Correrla con los mismos predictores y conjuntos de activaciones que en el punto b), y comparar. ¿En qué casos conviene evitar el salto? ¿Por qué el resultado de esta versión no depende de las activaciones?

d) Correr las dos versiones en el procesador out-of-order, con ambos conjuntos de activaciones, y comparar con los resultados de los puntos b) y c). ¿En qué casos el procesador out-of-order aprovecha su mayor ancho de emisión y en cuáles no? Usar Konata para justificarlo.

En el informe se deben incluir las tablas necesarias para que queden detallados todos los resultados, y los gráficos y capturas de los visualizadores necesarios para explicar y justificar el funcionamiento en los puntos a) a d).